

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)
Department of Computer Science and Engineering
CO1 ASSESSMENT TOOL 2 – Agile Sprint Planning & User Story Workshop

Course Code:	CSA10 / CSA1023	Course Title:	Software Engineering
CO Assessed:	CO1	Assessment:	Assessment Tool 2 – Agile Sprint Planning & User Story Workshop
Weightage:	20%	Total Marks:	25
CO1 Statement:	<i>Apply advanced methodologies and agile project management techniques to manage software projects effectively</i>		
Student Name:	Beffy A Shanika	Reg. No.:	192511387
Date:	09/10/2026	Duration:	60 minutes

Code of Conduct

I Beffy A Shanika(192511387) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: beffy

Annexure A – Agile Sprint Planning & User Story Workshop

Instructions to Students:

Each student is assigned an individual software project problem statement. Based on the given problem statement, perform an **Agile Sprint Planning and User Story Workshop** by applying Agile software engineering practices. Prepare a comprehensive sprint planning document covering the following aspects:

Based on the given problem statement, perform an **Agile Sprint Planning and User Story Workshop** by applying Agile software engineering practices. Prepare a comprehensive sprint planning document covering the following aspects:

- 1. Identify the Project Vision and Stakeholders**
 - Define the project's vision, objectives, and identify all key stakeholders involved.
- 2. Create Epics and User Stories**
 - Break down the requirements into Epics and formulate well-structured User Stories using the format:
As a <user>, I want <feature>, so that <benefit>.
- 3. Define Acceptance Criteria**
 - Write clear and measurable acceptance criteria for each selected user story using the Given–When–Then format or equivalent.
- 4. Prioritize the Product Backlog**
 - Organize and prioritize the product backlog based on business value, customer needs, and project dependencies.
- 5. Plan the Sprint Backlog**
 - Select high-priority user stories for the first sprint and divide them into actionable development tasks.
- 6. Estimate Effort Using Story Points**
 - Assign story points to each selected user story using an appropriate estimation technique (e.g., Fibonacci sequence or Planning Poker) and justify your estimates.
- 7. Present the Sprint Goal and Expected Deliverables**
 - Define a clear sprint goal and describe the expected sprint deliverables, including the features to be completed at the end of the sprint.

Agile Sprint Planning and User Story Workshop

Project Title: Legacy-to-Microservices Modernization and Technical-Debt Recovery Platform

1. Project Vision and Stakeholders

1.1 Project Vision

The vision of this project is to develop a platform that helps organizations modernize their legacy software systems into microservices gradually, safely, and efficiently. The platform will identify tightly coupled modules, technical debt, outdated dependencies, and high-risk components. It will support API-based migration, automated testing, containerization, CI/CD, monitoring, and rollback mechanisms. By following Agile practices, the project aims to reduce migration risks, maintain business continuity, and improve software maintainability, scalability, and reliability.

1.2 Project Objectives

- Identify technical debt and tightly coupled modules in legacy applications.
- Identify suitable components for incremental migration.
- Support API-based integration and microservices development.
- Automate regression and integration testing.
- Use containers and CI/CD pipelines for deployment.
- Monitor application performance and migration failures.
- Provide rollback facilities to recover from failed deployments.
- Measure modernization progress using software quality metrics.

1.3 Key Stakeholders

Stakeholder	Role and Responsibility
Product Owner	Defines requirements and prioritizes the product backlog.
Scrum Master	Coordinates sprint activities and removes obstacles.
Development Team	Analyses, refactors, and develops software components.
Software Architect	Designs the microservices architecture and migration strategy.

QA/Test Engineer	Performs automated regression and integration testing.
DevOps Engineer	Manages containers, CI/CD pipelines, and deployment.
System Administrator	Monitors system health, availability, and performance.
Business Users	Ensure that existing business functions continue to work correctly.
Project Sponsor/Management	Reviews project progress, cost, and business benefits.

2. Create Epics and User Stories

Epics represent large areas of functionality. Each epic is divided into smaller user stories that can be implemented and tested during individual sprints.

2.1 Identified Epics

Epic ID	Epic Name	Description
E1	Legacy Application Analysis	Analyse source code, dependencies, and application structure.
E2	Technical Debt Identification	Identify code complexity, outdated dependencies, and maintenance issues.
E3	Migration Planning	Select suitable components for incremental modernization.
E4	API and Microservices Development	Separate selected components and expose them through APIs.
E5	Automated Testing	Validate migrated services through regression and integration tests.
E6	Containerization and CI/CD	Package services and automate build and deployment processes.
E7	Monitoring and Rollback	Track performance, detect failures, and support recovery.
E8	Progress Reporting and Documentation	Measure modernization improvements and document changes.

2.2 User Stories

Story ID	Epic	User Story
US01	E1	As a developer, I want to upload a legacy application, so that I can analyse its structure.
US02	E1	As a software architect, I want to identify tightly coupled modules, so that I can plan component separation.
US03	E2	As a developer, I want to identify technical debt and obsolete dependencies, so that I can prioritize maintenance work.
US04	E3	As a software architect, I want to rank components by migration risk and business value, so that I can select suitable migration candidates.
US05	E4	As a developer, I want to expose selected components through APIs, so that they can communicate with other services.
US06	E4	As a developer, I want to migrate selected components into microservices, so that the application becomes easier to maintain and scale.
US07	E5	As a QA engineer, I want to execute automated regression tests, so that existing business functions continue to work correctly.
US08	E5	As a QA engineer, I want to run integration tests, so that communication between migrated services can be validated.
US09	E6	As a DevOps engineer, I want to package microservices in containers, so that they can run consistently across environments.
US10	E6	As a DevOps engineer, I want to automate builds and deployments through CI/CD, so that releases become faster and more reliable.
US11	E7	As a system administrator, I want to monitor service health and performance, so that I can detect migration-related failures.
US12	E7	As a DevOps engineer, I want to roll back a failed deployment, so that service disruption can be minimized.
US13	E8	As a project manager, I want to view modernization progress and quality metrics, so that I can measure project improvements.
US14	E8	As a developer, I want to maintain migration documentation and version history, so that changes can be traced and maintained.

3. Define Acceptance Criteria

Acceptance criteria describe the conditions that must be satisfied for a user story to be considered complete. The following examples use the Given–When–Then format.

US01: Upload a Legacy Application

- Given that the user is logged into the platform, when the user uploads a supported project archive, then the system should validate the file and display a successful upload message.
- Given that the uploaded file is invalid or unsupported, when the upload is submitted, then the system should display an appropriate error message.
- Given that a valid application is uploaded, when the upload finishes, then the application should be available for analysis.

US02: Identify Tightly Coupled Modules

- Given that a legacy application has been uploaded, when dependency analysis is executed, then the system should generate a module dependency report.
- Given that dependencies exist between modules, when the report is generated, then the system should identify modules with high dependency counts.
- Given that analysis is complete, when the user opens the report, then the results should include module names and their identified dependencies.

US03: Identify Technical Debt

- Given that source code is available, when static analysis is performed, then the system should identify supported code-quality issues.
- Given that outdated dependencies are detected, when the analysis report is generated, then the affected dependency names and available version information should be displayed.
- Given that technical debt findings exist, when the report is viewed, then findings should be categorized by severity.

US04: Rank Migration Candidates

- Given that module analysis results are available, when the migration assessment is executed, then the system should rank candidate modules using defined risk and business-value criteria.

- Given that a module has high coupling or critical business importance, when its migration score is calculated, then these factors should be reflected in the assessment.
- Given that the assessment is complete, when the architect views the results, then each candidate should have a score and recommendation.

US05: Expose Components Through APIs

- Given that a component is selected for migration, when an API endpoint is configured, then the endpoint should return the expected response for a valid request.
- Given that an invalid request is received, when the API processes it, then the system should return an appropriate error response.
- Given that an API is available, when its documentation is opened, then the supported endpoints and request formats should be visible.

US07: Automated Regression Testing

- Given that regression test cases have been configured, when the test suite is executed, then the system should display passed and failed test counts.
- Given that a previously working business function fails a test, when the results are generated, then the failure should be reported.
- Given that the test execution is complete, when the report is viewed, then the results should include the test status and failure details.

US09: Containerize Microservices

- Given that a migrated service has a valid container configuration, when the container image is built, then the build should complete successfully.
- Given that the image is available, when the service is started in the configured environment, then the service should become accessible.
- Given that the container fails to start, when the deployment is checked, then the failure should be recorded in the logs.

US10: Automate CI/CD

- Given that the project is connected to a version-control repository, when a code change is committed, then the configured pipeline should trigger automatically.
- Given that the build or required tests fail, when the pipeline runs, then deployment should be blocked.
- Given that the build and required tests pass, when the pipeline completes, then the deployment stage should follow the configured release rules.

US11: Monitor Service Performance

- Given that a migrated service is running, when monitoring data is collected, then the dashboard should display available health and performance metrics.
- Given that a configured threshold is exceeded, when the monitoring rule is evaluated, then an alert should be generated.
- Given that a service failure occurs, when the failure is detected, then the event should be recorded.

US12: Roll Back a Failed Deployment

- Given that a previous stable release is available, when an authorized user initiates rollback, then the system should restore the previous release according to the configured procedure.
- Given that rollback is completed, when service health is checked, then the system should report the recovery status.
- Given that rollback fails, when the operation ends, then the failure should be logged and reported.

Note: The remaining stories, including progress reporting, documentation, and integration testing, should use equivalent acceptance criteria before implementation.

4. Prioritize the Product Backlog

The product backlog is prioritized according to business value, risk reduction, dependencies, and implementation needs. The MoSCoW technique is used to classify requirements as Must Have, Should Have, or Could Have.

Priority	Story ID	User Story Summary	Priority Reason
1	US01	Upload legacy application	Required before analysis can begin.
2	US02	Identify tightly coupled modules	Supports architectural analysis and migration planning.
3	US03	Identify technical debt	Helps identify maintenance risks.
4	US04	Rank migration candidates	Supports safe, incremental migration.
5	US07	Automate regression testing	Protects existing business functionality.
6	US05	Expose components through APIs	Enables communication between services.
7	US06	Migrate components into microservices	Delivers the core modernization capability.
8	US09	Containerize services	Supports consistent deployment.
9	US08	Perform integration testing	Validates interactions between services.
10	US10	Automate CI/CD	Improves release speed and reliability.
11	US11	Monitor performance	Helps detect operational problems.
12	US12	Roll back failed deployments	Reduces deployment risk and downtime.
13	US14	Maintain documentation and version history	Supports traceability and maintenance.
14	US13	Display modernization metrics	Demonstrates progress and quality improvements.

Priority Classification

- Must Have: US01, US02, US03, US04, US05, US07.
- Should Have: US06, US08, US09, US10, US11, US12.
- Could Have: US13, US14, depending on project time and available resources.

The ordering above is an initial plan. Dependencies may require some stories to be developed or tested together.

5. Plan the Sprint Backlog

5.1 Sprint Details

Item	Description
Sprint Number	Sprint 1
Sprint Duration	2 weeks
Sprint Theme	Legacy Application Analysis
Sprint Objective	Build the initial application upload and analysis workflow.
Development Approach	Agile Scrum
Sprint Participants	Product Owner, Scrum Master, Developers, QA Engineer

5.2 Selected Sprint Stories and Tasks

Story ID	Development Tasks	Responsible Role
US01	Design upload interface; implement file validation; store project metadata; test upload success and failure.	Developer
US02	Identify supported source formats; implement module/dependency analysis; generate dependency report; test analysis output.	Developer
US03	Configure static code analysis; detect supported quality issues; identify outdated dependencies where tooling permits; generate findings report.	Developer
US04	Define migration scoring criteria; implement an initial ranking method; display candidate scores; validate ranking with sample applications.	Architect/Developer
US07	Define baseline regression tests; implement initial test execution; capture results; display passed and failed test counts.	QA Engineer/Developer

5.3 Sprint Activities

Day/Period	Activity
Day 1	Sprint planning, requirement review, and task allocation.
Days 2–4	Implement application upload and validation.
Days 5–6	Develop module dependency and technical-debt analysis.
Days 7–8	Implement migration-candidate ranking and regression-test reporting.
Day 9	Integrate modules and perform system testing.
Day 10	Sprint review, demonstration, and retrospective.

This schedule is indicative. The team may adjust it according to task complexity and progress.

6. Estimate Effort Using Story Points

Story points estimate relative effort, complexity, and uncertainty rather than exact working hours. The Fibonacci sequence is used: 1, 2, 3, 5, 8, 13.

Planning Poker can be used by team members to discuss each story and agree on an estimate.

Story ID	User Story	Story Points	Justification
US01	Upload legacy application	3	Requires upload interface, validation, and basic error handling.
US02	Identify tightly coupled modules	8	Requires dependency analysis and report generation.
US03	Identify technical debt	8	Involves code-quality analysis and dependency checks.
US04	Rank migration candidates	5	Requires scoring rules and candidate ranking.
US07	Automated regression testing	5	Requires test execution and result reporting.
	Total	29	Initial Sprint 1 estimate

Estimation Justification

- 3 points: A relatively small feature with limited complexity.
- 5 points: Moderate complexity with several connected tasks.
- 8 points: Higher complexity, analysis challenges, and integration uncertainty.

The total of 29 points represents the estimated effort for the selected stories. It should be compared with the team's actual capacity and previous sprint velocity. If the team is new and has no established velocity, it may be safer to reduce the initial sprint scope.

7. Sprint Goal and Expected Deliverables

7.1 Sprint Goal

The goal of Sprint 1 is to develop an initial working platform that accepts a legacy application, analyses its module dependencies and technical debt, identifies suitable migration candidates, and reports baseline regression-test results.

7.2 Expected Deliverables

At the end of Sprint 1, the team expects to deliver:

1. A basic application upload interface with file validation.
2. An initial module dependency analysis feature.
3. A technical-debt report showing supported code-quality findings.
4. A migration-candidate ranking feature based on defined criteria.
5. A basic regression-test execution and reporting feature.
6. A version-controlled code repository containing the implemented modules.
7. Test cases and test results for the completed user stories.
8. Initial technical documentation describing the workflow and design.
9. A demonstration of the working prototype during the sprint review.

7.3 Definition of Done

A user story is considered complete when:

- Its acceptance criteria are satisfied.
- The implementation is reviewed and integrated into the main codebase.
- Relevant unit and integration tests pass.
- Known errors and limitations are documented.
- The feature is demonstrated to the Product Owner.
- The related documentation is updated.

8. Conclusion

The Agile Sprint Planning and User Story Workshop provides a structured approach to developing the Legacy-to-Microservices Modernization and Technical-Debt Recovery Platform. The project requirements are divided into epics and user stories, with measurable acceptance criteria and prioritized backlog items. Sprint 1 focuses on application upload, dependency analysis, technical-debt identification, migration-candidate ranking, and regression-test reporting. Story points help the team estimate relative effort, while the sprint goal and Definition of Done provide clear completion criteria. This approach supports incremental development, early feedback, risk reduction, and continuous improvement while preparing the platform for later API migration, containerization, CI/CD, monitoring, and rollback capabilities.

Rubrics for Calculation

Criteria	Excellent (5 Marks)	Good (3 Marks)	Satisfactory (2 Marks)	Needs Improvement (1 Mark)
Project Vision & Stakeholder Identification (30%)	Project vision is clear, comprehensive, and aligned with the problem statement. All relevant stakeholders and their roles are correctly identified.	Vision is clear with minor omissions. Most stakeholders are correctly identified.	Vision is partially defined. Some stakeholders are missing or incorrectly identified.	Vision is unclear or incomplete. Stakeholders are poorly identified or missing.
Epics & User Stories (25%)	Epics are well-defined and user stories are complete, follow Agile format, and fully capture functional requirements.	Epics and user stories are mostly complete with minor formatting or requirement issues.	User stories are partially complete with limited Agile structure.	User stories are incomplete, unclear, or do not follow Agile practices.
Acceptance Criteria (20%)	Acceptance criteria are specific, measurable, testable, and aligned with all user stories.	Acceptance criteria are mostly clear and testable with minor gaps.	Acceptance criteria are partially defined and lack clarity or completeness.	Acceptance criteria are missing or not aligned with user stories.
Product Backlog Prioritization & Sprint Planning (15%)	Product backlog is logically prioritized, sprint backlog is realistic, and task breakdown is complete.	Backlog prioritization and sprint planning are mostly appropriate with minor issues.	Prioritization or sprint planning is partially complete with limited justification.	Backlog prioritization and sprint planning are poorly organized or incomplete.
Story Point Estimation, Sprint Goal & Presentation (10%)	Story point estimation is well justified, sprint goal is clear, deliverables are realistic, and presentation is professional and well organized.	Estimation and sprint goal are mostly appropriate with minor inconsistencies. Presentation is clear.	Estimation or sprint goal lacks justification. Presentation is adequate but incomplete.	Estimation is unrealistic or missing. Sprint goal, deliverables, and presentation are unclear or incomplete.

Criteria	Mark
Project Vision & Stakeholder Identification (30%)	/5
Epics & User Stories (25%)	/5
Acceptance Criteria (20%)	/5
Product Backlog Prioritization & Sprint Planning (15%)	/5
Story Point Estimation, Sprint Goal & Presentation (10%)	/5
TOTAL	/25